Compiler Construction Project

Spring 2026

Compiler for a C-Like Language Subset

VM-Based Code Generation and Runtime Execution

Submitted by

Name	**CMS ID**
Aqsa Batool	*413777*
Muhammad Hamza	*407251*
Ahmed Mohiuddin Shah	*415216*

Project Report
May 8, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Changes Asked for During the Presentation **2**

4 Implemented Language Subset **3**

5 Project Structure **5**

6 Compiler Pipeline Overview **5**

7 Lexical Analysis **6**

8 Grammar Specification **6**

9 Parser Design and AST **8**

10 Semantic Analysis **9**

11 Intermediate Representation **10**

12 Object Code Generation **11**

13 VM Runtime Design **11**

14 Build and Execution **12**

15 Sample Programs and Outputs **12**

16 Testing Strategy **15**

17 Innovation Features **16**

18 Limitations and Future Work **17**

19 Screenshot **17**

20 Conclusion **17**

1 Introduction

This project implements a compiler for a well-defined subset of the C language. The implementation covers the complete frontend and execution pipeline: lexical analysis, syntax analysis, semantic analysis, intermediate representation, object-code generation, and runtime execution. Instead of generating native machine code, the compiler emits a custom human-readable VM object format (".vmo"), and a separate runtime executes that object code. This design keeps the compilation phases explicit and easy to demonstrate.

The project is organized around two tools:

- **cforge**: compiler executable that tokenizes, parses, checks, lowers, and emits VM object code.
- **crun**: runtime executable that loads the generated object file and executes it on a custom virtual machine.

The project also includes phase-wise inspection flags so that every important stage can be shown independently during presentation or debugging.

2 Objectives

The main objectives achieved in this project are:

1. Design a handwritten compiler for a meaningful subset of C.
2. Implement lexical analysis with position-aware tokens.
3. Implement recursive-descent parsing and explicit abstract syntax tree construction.
4. Perform semantic analysis including scope handling, type checking, function checking, and array checking.
5. Generate an intermediate representation and lower it to VM object code.
6. Execute compiled programs through a separate runtime.
7. Provide clean build, test, compile, and run workflows.

3 Changes Asked for During the Presentation

During the presentation, an additional lexer-level validation was requested for malformed numeric tokens that look like identifiers. The lexer was updated to reject these cases immediately instead of allowing them to continue into later stages.

Here is a program along with its output after the changes.

```c
int main()
{
    int 1fibs [10];
    int i = 0;
    while (i < 10)
    {
        int result = fib(i);
        fibs[i] = result;
        i = i + 1;
    }
```

Figure 1: A program with a lexer error where an identifier starts with a number.

```
hamza  .../compiler  main !?  v1.24.5  10:49
→ make compile FILE=samples/function_fib.c
cforge: lex: 17:9: invalid numeric token "1fibs"
make: *** [Makefile:28: compile] Error 1
```

Figure 2: A screenshot of lexer error output displayed in the terminal.

4 Implemented Language Subset

The implemented subset is intentionally smaller than full ISO C, but large enough to demonstrate all core compiler phases.

Types

- `int`
- `char`
- `float`
- `void`

Declarations

- global scalar declarations
- global 1D fixed-size array declarations
- local scalar declarations
- local 1D fixed-size array declarations
- scalar initializers
- `char s[N] = "literal"`

Statements

- block statements
- assignments
- expression statements
- `if` / `else`
- `while`
- `return`

Expressions

- identifiers
- integer, float, char, and string literals
- function calls
- array indexing
- unary `-` and `!`
- binary `+ - * / % < <= > >= == != && ||`

Functions

- user-defined functions
- typed parameters
- return values
- `main`
- function must be defined before call

Builtins

- `print_int(int)`
- `print_float(float)`
- `print_char(char)`
- `print_str(char[])`
- `print_newline()`

Current Deliberate Limits

- no `for` statement
- no 2D arrays
- no array parameters
- no optimizer pass yet
- no escape sequences in char and string literals yet
- `&&` and `||` evaluate both operands

5 Project Structure

```
compiler/
|-- cmd/
|   |-- cforge/          # compiler entry point
|   `-- crun/            # runtime entry point
|-- internal/
|   |-- token/           # token kinds and positions
|   |-- lexer/           # handwritten scanner
|   |-- ast/             # AST node definitions and pretty printer
|   |-- parser/          # recursive-descent parser
|   |-- sema/            # semantic analysis and symbol checking
|   |-- ir/              # intermediate representation and lowering
|   |-- vm/              # VM object model, loader, and executor
|   |-- driver/          # compiler CLI logic
|   `-- runtimecli/      # runtime CLI logic
|-- samples/
|   |-- pass/            # valid sample programs
|   `-- fail/            # invalid sample programs
|-- Makefile
|-- README.md
`-- report.tex
```

6 Compiler Pipeline Overview

The implemented compilation and execution flow is:

```
source.c
  -> lexer
  -> tokens
  -> parser
  -> AST
  -> semantic analysis
  -> IR
  -> VM object code (.vmo)
  -> crun runtime
  -> observed program output
```

This separation is useful for both grading and debugging because each stage can be displayed independently using compiler flags.

7 Lexical Analysis

The lexer is handwritten in Go. It reads the source character stream and emits a sequence of tokens with line and column positions. These positions are reused by the parser and semantic analyzer to report precise diagnostics.

The scanner recognizes:

- keywords: `int`, `char`, `float`, `void`, `if`, `else`, `while`, `return`
- identifiers
- integer literals
- float literals
- character literals
- string literals
- operators and punctuation
- single-line comments beginning with `//`

The compiler intentionally supports only plain character and string literals in this version, with no escape-sequence processing.

Example token dump produced by:

```
./bin/cforge --tokens samples/slice7_run.c
```

```
1:1 KW_INT        "int"
2:5 KW_CHAR       "char"
2:14 CHAR_LIT      "'A'"
5:15 STRING_LIT    "\"hello\""
9:1 EOF           ""
```

8 Grammar Specification

The grammar is implemented with handwritten recursive-descent parsing. The following EBNF describes the accepted subset.

Top-Level Declarations

```
program         = { top_decl } EOF ;

top_decl        = var_decl | function_def ;

function_def    = type_spec IDENTIFIER "(" params ")" block ;

params          = "void"
                | param { "," param }
                | empty ;

param           = type_spec IDENTIFIER ;

var_decl        = type_spec IDENTIFIER [ array_spec ] [ "=" expr ] ";"
    ;

array_spec      = "[" INT_LITERAL "]" ;

type_spec       = "int" | "char" | "float" | "void" ;
```

Blocks and Statements

```
block           = "{" { local_decl } { stmt } "}" ;

local_decl      = var_decl ;

stmt            = block
                | if_stmt
                | while_stmt
                | return_stmt
                | expr_stmt ;

if_stmt         = "if" "(" expr ")" stmt [ "else" stmt ] ;

while_stmt      = "while" "(" expr ")" stmt ;

return_stmt     = "return" [ expr ] ";" ;

expr_stmt       = expr ";" ;
```

Expressions

```
expr            = assignment ;

assignment      = logical_or [ "=" assignment ] ;

logical_or      = logical_and { "||" logical_and } ;

logical_and     = equality { "&&" equality } ;

equality        = relational { ("==" | "!=") relational } ;
```

```
relational     = additive { ("<" | "<=" | ">" | ">=") additive } ;

additive       = multiplicative { ("+" | "-") multiplicative } ;

multiplicative = unary { ("*" | "/" | "%") unary } ;

unary          = ("-" | "!") unary | postfix ;

postfix        = primary
                 { "(" [ arguments ] ")"
                 | "[" expr "]" } ;

arguments      = expr { "," expr } ;

primary        = IDENTIFIER
               | INT_LITERAL
               | FLOAT_LITERAL
               | CHAR_LITERAL
               | STRING_LITERAL
               | "(" expr ")" ;
```

This grammar is enough to express the required subset while remaining simple enough for a handwritten parser.

9 Parser Design and AST

The parser is implemented as a recursive-descent parser. This decision was made for three reasons:

1. the language subset is moderate and LL-friendly after factoring expression precedence into separate functions,
2. parser logic remains easy to debug and explain during presentation,
3. explicit AST construction is useful because the project must expose each stage separately.

Top-level declarations are parsed by first reading a type and identifier, then deciding whether the declaration is a function definition or a global variable. Blocks enforce the simplified "declarations first, then statements" rule.

The AST stores:

- top-level declaration list
- separate lists of global variables and functions
- statements such as blocks, declarations, conditionals, loops, returns, and expression statements
- expressions such as literals, identifiers, calls, array indexing, unary operators, binary operators, and assignments

Example AST excerpt for a global array program:

```
Program
    GlobalVarDecl name=nums type=int array=3
    FuncDecl name=main return=int
```

```
        Params <empty>
        Body
            BlockStmt
                ExprStmt
                    AssignExpr
                        Target
                            IndexExpr name=nums
                                IntLiteral value=0
```

10 Semantic Analysis

Semantic analysis is the stage that turns a syntactically valid program into a semantically meaningful one. The project checks declarations, visibility, assignment compatibility, function-call correctness, return types, and array usage.

Symbol Tables and Scope

The analyzer uses:

- a global symbol table for global variables,
- a function table for user-defined functions and builtins,
- a stack of local scopes for nested blocks.

Lookup order is:

1. innermost local scope,
2. outer local scopes,
3. global scope.

This correctly supports shadowing. For example, a local variable named `g` hides a global variable `g` inside the block where the local variable is declared.

Checks Implemented

The analyzer reports errors for:

- duplicate global declarations
- duplicate local declarations in the same scope
- duplicate function declarations
- global/function name collisions
- undeclared identifiers
- using a non-array as an array
- wrong function argument count
- wrong argument types
- invalid assignment targets

- illegal narrowing such as `float -> int`
- invalid global or local initializers
- mismatched return values

Type Rules

Safe widening conversions are allowed:

- `char -> int`
- `char -> float`
- `int -> float`

Unsafe narrowing conversions are rejected:

- `float -> int`
- `int -> char`
- `float -> char`

Comparison and logical expressions produce an `int` result with value `0` or `1`. Conditions in `if` and `while` must be numeric scalar expressions.

Builtins

The following builtin signatures are pre-registered before user-defined functions are analyzed:

```
print_int(int)
print_float(float)
print_char(char)
print_str(char[])
print_newline()
```

11 Intermediate Representation

After semantic analysis, the compiler lowers the AST to a custom intermediate representation. This IR is already close to the VM instruction set, but still expressed in compiler-owned data structures. The IR contains:

- explicit global declarations
- explicit global initialization instructions
- function-level instructions
- jumps and labels for control flow
- function-call instructions
- local/global load and store instructions
- local/global array indexing instructions

For example, storing into a global array lowers into explicit indexed store instructions rather than remaining as a high-level AST node.

12 Object Code Generation

The compiler executable `cforge` lowers IR into a human-readable object format with extension `.vmo`. This object code is intentionally textual instead of binary because it is easier to debug, diff, inspect, and present.

The object file format has three sections:

1. `globals ... endglobals`
2. `init ... endinit`
3. one or more `func ... end` blocks

Example excerpt:

```
globals
  msg:char[6]
endglobals
init
  INIT_STRING msg "hello"
endinit
func main return=int
  params <empty>
  PUSH_GLOBAL_REF msg
  CALL_BUILTIN print_str 1
  CALL_BUILTIN print_newline 0
  PUSH_INT 0
  RET
end
```

This design gives a clear boundary between compiler and runtime, which also helps the final demo because the generated object code can be shown directly.

13 VM Runtime Design

The runtime executable `crun` parses the `.vmo` object file and executes it on a custom virtual machine.

Runtime Memory Model

The VM uses:

- a global storage map for global variables and arrays,
- a frame-local storage map for each function activation,
- a typed value stack for expression evaluation.

Each value carries a kind such as `int`, `float`, `char`, string literal, or array reference. Arrays are stored as fixed-size homogeneous cells, and `print_str` can accept either a string literal or a reference to a `char[]` object.

Execution Flow

The runtime performs the following steps:

1. allocate global storage from the `globals` section,
2. execute the `init` section once,
3. find `main`,
4. execute `main` using local frames and the evaluation stack.

Builtins in the Runtime

Builtins are implemented inside the VM rather than being linked externally. This keeps the system self-contained and avoids dependence on platform-specific calling conventions.

14 Build and Execution

The project can be built from the repository root with:

```
make build
```

The automated tests can be run with:

```
make test
```

Compilation to object code:

```
./bin/cforge samples/slice9_run.c -o program.vmo
```

Runtime execution:

```
./bin/crun program.vmo
```

Phase-wise output can be obtained with:

```
./bin/cforge --tokens  samples/slice8_run.c
./bin/cforge --ast     samples/slice8_run.c
./bin/cforge --sema    samples/slice8_run.c
./bin/cforge --ir      samples/slice8_run.c
./bin/cforge --codegen samples/slice8_run.c
```

The Makefile also provides convenience targets such as:

```
make run FILE=samples/slice9_run.c OBJ=program.vmo
make phases FILE=samples/slice8_run.c OBJ=program.vmo
```

15 Sample Programs and Outputs

To keep the report compact while still representative, four valid programs and one invalid program are highlighted below. The repository contains a larger pass and fail matrix used during testing.

Program 1: Globals, Shadowing, and Function Use

File: `samples/pass/global_scalar.c`

```
int g = 10;

int add(int x) {
    return x + g;
}

int main() {
    print_int(add(5));
    print_newline();
    return 0;
}
```

This demonstrates:

- global variable declaration
- user-defined function call
- global lookup inside a function body

Observed output:

```
15
```

Program 2: Arrays and Strings

File: `samples/slice8_run.c`

```
int main() {
    char s[6] = "hello";
    int a[3];
    a[0] = 4;
    a[1] = 5;
    a[2] = a[0] + a[1];
    print_str(s);
    print_newline();
    print_int(a[2]);
    print_newline();
    return a[2];
}
```

This demonstrates:

- fixed-size arrays
- `char[]` string initialization
- array indexing and assignment
- builtin string and integer printing

Observed output:

```
hello
9
```

Program 3: Float Arithmetic and Function Return

File: `samples/slice9_run.c`

```c
float addf(float a, float b) {
    return a + b;
}

int main() {
    float x = 2.5;
    float y = 4.0;
    float z = addf(x, y);
    print_float(z);
    print_newline();
    return 0;
}
```

This demonstrates:

- float declarations and literals
- float-returning function
- builtin float printing

Observed output:

```
6.5
```

Program 4: Control Flow

File: `samples/slice5_run.c`

```c
int main() {
    int i = 0;
    int sum = 0;

    while (i < 5) {
        if (i == 3) {
            print_int(99);
            print_newline();
        }

        sum = sum + i;
        i = i + 1;
    }

    print_int(sum);
    print_newline();
    return sum;
}
```

This demonstrates:

- `while` loop
- nested `if`
- jump-based lowering to the VM

Observed output:

```
99
10
```

Program 5: Invalid Narrowing Conversion

File: `samples/fail/float_to_int.c`

```c
int main() {
    int x = 2.5;
    return x;
}
```

Observed compiler diagnostic:

```
cforge: semantic: initializer type mismatch for x at 2:5: want int
```

This example demonstrates that the compiler rejects implicit narrowing from `float` to `int`.

16 Testing Strategy

Testing was performed at three levels.

Unit Tests

Go unit tests exist for:

- tokenization
- parsing of declarations, functions, arrays, and expressions
- semantic analysis of declarations, calls, scopes, and type rules
- VM execution, jumps, function calls, object-format round trips, and global initialization

Sample Matrix

The project contains separate `samples/pass` and `samples/fail` directories. Valid samples confirm successful execution, while invalid samples confirm that diagnostics are reported. The error handling is currently *fail-fast*: one run reports the first encountered semantic or syntax error. For this reason, the negative test suite uses multiple distinct invalid sample programs rather than a single file containing many unrelated problems.

Phase Verification

Important samples were checked through:

1. token dump
2. AST dump
3. semantic pass
4. IR dump
5. object-code dump
6. full runtime execution

This phase-by-phase validation helped ensure that bugs were isolated at the correct stage rather than being observed only at the final execution step.

17 Innovation Features

The project satisfies the core assignment requirements and also includes innovation-oriented work beyond the minimum requirements.

Core Requirements Completed

- lexical analysis
- syntax analysis
- semantic analysis
- global and local scope management
- intermediate representation
- code generation and executable runtime path

Innovation Features Implemented

1. **Implementation in Go instead of C/C++**
2. **Array support**. The compiler supports 1D fixed-size arrays of `int`, `char`, and `float`, including indexing and string initialization for `char[]`.
3. **Custom target-code path**. Instead of stopping at IR, the compiler emits a standalone object-code format executed by a separate custom runtime. This demonstrates a complete end-to-end design beyond the minimum frontend-only approach.

These additions strengthen the project in terms of completeness, maintenance, and modularity.

18 Limitations and Future Work

The current implementation is complete for the chosen subset, but several extensions remain possible.

- add `for` loops
- add 2D arrays
- add array parameters
- add an optimizer pass such as constant folding or dead code removal
- add escape-sequence support in char and string literals
- implement true short-circuit semantics for `&&` and `||`
- add global constant propagation or other global optimizations

These are natural next steps, but were intentionally deferred to keep the core compiler stable and demonstrable by the deadline.

19 Screenshot

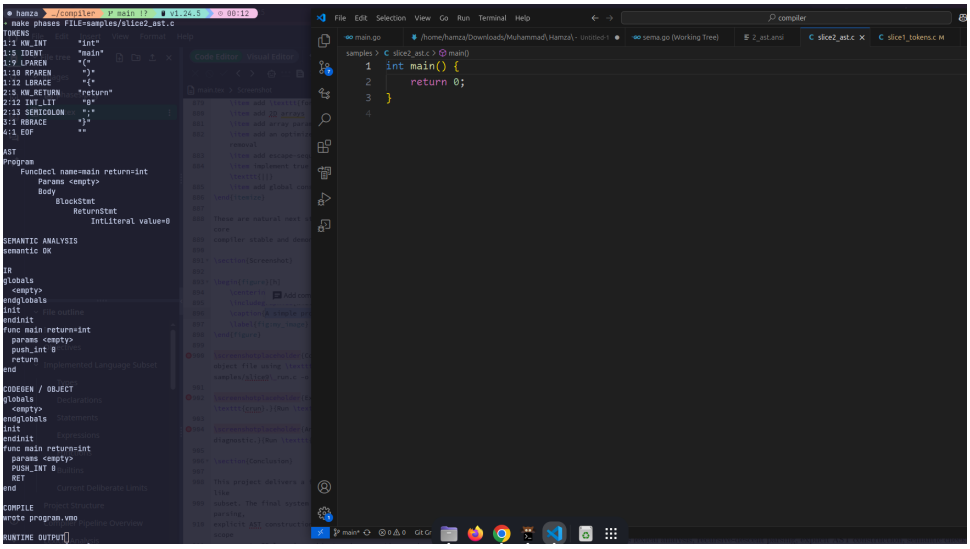

Figure 3: A program compiled with our compiler showing all of the compilation phases.

20 Conclusion

This project delivers a full compiler pipeline for a carefully chosen C-like subset. The final system includes lexical analysis, recursive-descent parsing, explicit AST construction, semantic checking with both global and local scope management, IR lowering, object-code generation, and runtime execution on a custom VM. The use of separate compiler and runtime tools, together with array support and a non-C/C++ implementation language, also provides meaningful innovation beyond the core requirements. Overall, the implementation satisfies the main project goals while remaining modular, demonstrable, and extensible.